

Newborough Warren

Interactive Web Tools

Technical Note

Model Equations · Data Architecture · Rendering

Hollingham (2026)

June 2026

Part A

Groundwater Flooding Forecaster

A1. Overview

Document version — updated 2026-08-12.

The Groundwater Flooding Forecaster (forecaster.html) is a single-page application built from a Jinja-style template (forecaster_template.html) and a JSON data bundle injected by Script 11b (11b_spatial_thresholds.py, v1.6.3). From the May 2026 simplification onwards, the forecaster presents the report's cluster-block equations (Tables 6, 7, and 10) with inline value substitution. From v1.2.0 it additionally presents an ecohydrology block (EWI, tiered EbF, and live MSL5 — see A11). The per-well SSM iteration path (ssmlterate, horizonMonths, FORECAST_SOURCE tabs, and per-well SSM/P_flood coefficient exports) has been removed.

A2. Data Bundle Structure

Key	Contents
cluster_coeffs	Per-cluster: label, peak_month, trough_month, P_flood slope A and intercept B, P_clim_total_mm, horizon_months, monthly_clim (12-month depth climatology). Five clusters: C1–C5.
block_tf	Seasonal transfer-function coefficients (b1, b2, c, R²) for each geographic block, split into winter and summer sub-models. Each block lists its member clusters.
P_clim / PET_clim	Monthly climatological precipitation and PET (mm), keyed 1–12, from RAF Valley 2005–2026 means.
winter_climatology_mm	Mean Oct–Mar cumulative rainfall (~516 mm), the denominator for λ.
wells	Array of well objects with: name, display_name, E/N (OSGB), ground_elev, cluster, nearest_cluster_only, default_h_prev, default_h_max, and (from the simplification) monthly_clim (per-well 12-month depth climatology), trough_month, and peak_month.
base_layer	Map extent (OSGB: E 240100–243900, N 362100–365900 — a 3800 m × 3800 m square), Base64 hillshade PNG, and KML feature polylines with styling.

A3. Model Equations

The forecaster now presents only the three report-table equations. Per-well SSM iteration has been removed.

A3.1 Block transfer functions (Tables 6 and 7)

Forecast 1 (winter peak, Table 6):

$$h_{\text{peak}} = \beta_1 \cdot P_{\text{winter}} + \beta_2 \cdot h_{\text{min}}(\text{summer}) + \text{intercept}$$

Forecast 2 (summer minimum, Table 7):

$$h_{\min} = \beta_1 \cdot P_{\text{summer}} + \beta_2 \cdot h_{\max}(\text{winter}) + \text{intercept}$$

R² values range from 0.41 (Lake Edge summer) to 0.92 (Forest summer). When R² < 0.50 the forecast card displays a low-explanatory-power caveat. P_{winter} is the climatological Oct–peak_{month} total scaled by λ; P_{summer} is the climatological Apr–Sep total (always at λ = 1.00).

A3.2 P_{flood} linear form (Table 10)

Forecast 3 (cumulative rainfall to slack floor):

$$P_{\text{flood}} \text{ (mm)} = A \cdot d + B$$

where d is depth below ground (m, positive), A is slope (mm/m), and B is intercept (mm). The result is normalised as λ = P_{flood} / P_{clim_{total}}.

A4. Cluster and Block Mapping

Cluster	Label	Block TF	Peak	Horizon
C1	Lake Edge	Lake_Edge	Jan	4 mo
C2	Dune	Eastern_Block	Jan	4 mo
C3	Western Residual	Western_Block	Feb	5 mo
C4	Main Forest	Forest	Feb	5 mo
C5	Coastal Forest	Coastal_Forest	Feb	5 mo

A5. Per-Well Monthly Climatology

Script 11b reads 01_wells_clean.csv and computes a 12-month depth climatology per well (mean depth below ground for each calendar month). These are injected as monthly_{clim}, trough_{month}, and peak_{month} fields on each well object. The cluster_{coeffs} also receive their own monthly_{clim} (cluster-average depths), trough_{month}, and peak_{month}. The forecaster’s renderWellMeta function renders two stacked tables — per-well first, then cluster — each labelled by the entity’s own trough and peak months (which may differ).

A6. Two-Input Architecture

The sidebar contains two separate depth inputs, each driving a distinct forecast. This split resolves an ambiguity in earlier versions where a single “observed depth” field was silently used as both the summer minimum (for Forecast 1) and the current depth (for Forecast 3), which confused users whose today’s reading was not their summer minimum.

- current-depth-input: feeds Forecast 3’s d. Default: coeff.monthly_{clim}[currentMonth] (cluster’s long-term depth for the current calendar month), falling back to default_{h_{prev}} if no monthly climatology.
- summer-min-input: feeds Forecast 1’s h_{min}. Default: default_{h_{prev}} (cluster’s long-term summer minimum).

Both inputs fire a shared `onDepthInput` handler that calls `renderForecasts.selectWell()` populates both from cluster defaults when the user picks a well. Forecast 2 continues to use `default_h_max` (cluster-mean winter peak) with no user input.

A7. Timing Note Logic

Forecast 1 includes an amber timing note when the current calendar month falls before the cluster's `trough_month` (the month in which the summer minimum is typically reached). The `inWinterSeason()` function walks the calendar from `trough_month` towards March; if the current month is reached before March, the minimum has been observed. If not, the note reminds the user when this year's minimum is typically reached and that the summer-min input currently holds the long-term default.

A8. Live Met Office Integration

On initialisation, `loadLiveData()` fetches RAF Valley station data. The parser handles estimated values (asterisk-suffixed) and missing data ("---"). `buildLiveBanner()` computes Oct–Mar cumulative totals for current and prior hydrological winters, deriving λ = observed / climatological. The display adapts by calendar month (Oct–Dec emphasises the previous winter; Jan–Apr the current; May–Sep the most recently completed).

If the fetch fails, a manual fallback modal accepts pasted `valleydata.txt` and parses it identically.

A9. Map Rendering

The map uses a square SVG `viewBox` (1000 × 1000) matching the square base-layer extent (3800 m × 3800 m, OSGB E 240100–243900, N 362100–365900). The `project()` function linearly maps OSGB coordinates to SVG space with 30 px padding. `preserveAspectRatio` is set to `xMidYMin meet`, top-aligning the content. The legend overlays the top-left corner; the rainfall slider overlays the top-right.

Rendering layers: (1) Base64 hillshade PNG at 85% opacity, (2) KML polylines/polygons clipped to the plot area, (3) well dots ($r=10$, $r=14$ when selected) coloured by `P_flood` category, re-rendered on every slider change.

Note for maintainers: `MAP_W` and `MAP_H` control only the SVG `viewBox`, not the geographic extent. The geographic extent comes from `DATA.base_layer.extent`. If one changes without the other, `project()` independently stretches each axis, distorting the hillshade. Keep `viewBox` aspect = base-layer aspect. Currently both are square (1:1).

A10. Resizable Panels

The main layout uses CSS flex with 6 px draggable gutters between the sidebar, map, and forecast panel. `setupResizer()` attaches mouse and touch handlers; dragging sets the adjacent panel's flex-basis within a 200–900 px range. Widths persist via `localStorage` (keys `nb-panel-width-sidebar` and `nb-panel-width-panel`). On screens narrower than 960 px, panels stack vertically and gutters are hidden.

A11. Ecohydrology Block (EWI, EbF, MSL5)

From forecaster v1.2.0 the well-details panel carries an ecohydrology block alongside the cluster-block forecasts. Three indices are shown when the supporting feeds are present:

Equilibrium Wetness Index (EWI) — a dynamics-only index of the well's equilibrium wetness state (in metres below ground; a higher, shallower value means wetter), reconstructed from the well's state-space (SSM) coefficients and independent of MSL5. It is read from `forecaster_indices.json` and shown as an Equilibrium wetness index (EWI) row. Because it derives from the coefficients alone it needs a shorter record than a measured spring-level series; it is scoped to the open dune and degrades in the forest.

Ellenberg-F (EbF), tiered — a projected Ellenberg-F moisture value, also from `forecaster_indices.json`, tagged by tier (measured, projected, or extrapolated) to indicate how far it is inferred beyond the calibrated set.

Five-year mean spring level (MSL5) — the latest MSL5 for the well, read from the living hub feeds (`latest_readings.json`, `forecaster_msl5.json`) with a well → cluster → climatology fallback, and shown with an inline source / as-of label so the currency of the value is visible.

Feeds are optional. If they do not load (for example the standalone file opened offline) the block falls back to climatology exactly as before; when a feed is present the forecast month follows its as_of round (v1.1.1). The per-well row is skipped gracefully for extended-network wells whose records are too short to carry a climatology.

A12. Build Provenance

Script 11b assembles block transfer-function coefficients (Tables 6/7/10), cluster metadata, per-well monthly climatology (from `01_wells_clean.csv`), hillshade, KML features, and RAF Valley climatology into a single `.html` file from the `forecaster_template.html` template. Google Fonts (Libre Baskerville, Source Sans 3, JetBrains Mono) are the only external resource; the tool degrades gracefully without them.

Part B

Hydrological Scenario Viewer

B1. Overview

The Scenario Viewer (`scenario_viewer.html`) is generated by Script 19 (`19_spatial_groundwater.py`, v2.13.2). All computation is client-side JavaScript; no server requests are made after the initial page load.

B2. Physical Constants

Parameter	Value and source
Hydraulic conductivity K	6 m/day (Betson et al. 2002)
Forest interception (Corsican pine)	24% (Freeman 2008)
Broadleaf interception (deciduous)	15% annual mean (Komatsu et al. 2011)

Sy floor (C1 / C2–C5)	6% / 12%
Ridge mask threshold	1.0 m
Excluded wells	ceh12 (bedrock), ceh15 (forest slack edge)

B3. Δh Model Equation

For each well, baseline equilibrium net recharge is:

$$\mathbf{net_0 = b_1 \cdot P_{eff_0} - b_2 \cdot PET_0 - b_3 \cdot |h|}$$

Under a scenario:

$$\mathbf{net_{sc} = b_1 \cdot P_{eff_{sc}} - b_{2_{sc}} \cdot PET_{sc} - b_3 \cdot |h|}$$

where $P_{eff_{sc}} = P \times sP \times (1 - I_{scenario})$ for forest wells, $PET_{sc} = PET \times sPET$, and $b_{2_{sc}} = b_2 \times sB_2$ (where sB_2 is now season-specific: $sB2_w$ for winter, $sB2_s$ for summer). The per-well $\Delta h = net_{sc} - net_0$. For the annual season, Δh is the unweighted mean of winter and summer values (each computed with their respective seasonal sB_2). The b_3 drainage term cancels in the difference.

B4. Scenario Parameter Sets

From v2.6.0, the β_2 scaling is seasonally resolved: separate sB2_w (winter) and sB2_s (summer) multipliers replace the former per-cluster sB2_c4/sB2_c5 controls. Both are applied identically to C4 and C5.

Scenario	sP_w	sP_s	sPET_w	sPET_s	I_c4/c5	sB2_w	sB2_s
Baseline	1.00	1.00	1.00	1.00	0.24	1.00	1.00
UKCP18 2050s	1.10	0.85	1.05	1.20	0.24	1.00	1.00
UKCP18 2080s	1.20	0.70	1.10	1.35	0.24	1.00	1.00
Clearfell	1.00	1.00	1.00	1.00	0.00	per run*	per run*
Broadleaf	1.00	1.00	1.00	1.00	0.15	0.88†	1.08†
Thinning	1.00	1.00	1.00	1.00	0.12	per run*	per run*

* Clearfell and Thinning sB2 values are computed dynamically from the BACI-corrected Edge-tier β_2 ratio (Script 10e, loaded via `load_clearfell_b2_multiplier()` in `clearfell_common.py`). The clearfell multiplier is the full Edge-tier ratio; thinning is half the perturbation above unity. C5 receives the same multiplier as C4 by extrapolation. Exact values vary with each pipeline run.

† Broadleaf sB2 values derive from the Script 21 deciduous phenology profile: winter 0.88× reflects reduced transpiration during the leaf-off period, summer 1.08× reflects elevated transpiration during the growing season. This seasonal split is new in v2.6.0.

B5. Spatial Interpolation

The map surface uses inverse-distance-weighted (IDW) interpolation with power 1 and $k = 8$ nearest neighbours. A minimum distance floor of 10 m prevents singularities; points within 5 m of a well return the well's exact value. Interpolation is constrained to the site boundary polygon.

In depth mode, a DEM grid (160 × 110 cells) provides the ground surface. Depth = DEM elevation minus IDW head. Ridge masking suppresses rendering where DEM exceeds the IDW-interpolated DEM surface by > 1.0 m.

B6. Data Bundle

- WELLS: well objects with name, cluster, coordinates, seasonal heads, Sy, DEM ground elevation, and two coefficient sets — the comparison-window fit (b1, b2, b3, from 03_master_data.csv) and the whole-record fit (b1f, b2f, b3f, from 03_15_per_well_window_sensitivity.csv) — with the β_3 p-value on each basis (p3w, p3f).
- POLYS: KML polygons (site boundary, forest, clearfell, broadleaf, lake) and tracks, the eighteen linear features read from Features.kml as {name, pts} records.
- CLIMATE: seasonal baselines (P/PET), cluster heads, cluster betas, monthly climatology.
- DEM_GRID: downsampled elevation grid for ridge masking and depth mode.
- HILLSHADE: Base64 PNG (1100 × 750 px), site-masked, quantised to 64 grey levels and emitted opaque; the viewer blends it at draw time under the Basemap slider.

B7. Viewer Extent

Fixed at E 240200–243700, N 362400–364800 (OSGB36), covering all 77 wells. Canvas default 640 × 440 px, resizable. A parallel CSV (19_scenario_summary.csv) mirrors the viewer's calculations for the manuscript.

B8. Build Provenance

Script 19 assembles data from Scripts 01, 02, 03 (both 03_master_data.csv and 03_15_per_well_window_sensitivity.csv) and 18, together with the Script 10e β_2 multipliers loaded through clearfell_common and the site GIS layers — the DEM and the site boundary, clearfell, broadleaf restock and Features KML files. Script 26b's per-well projection table is read as a cross-check target, not as an input. Constants come from utils/config.py, including the cluster labels and the UKCP18 multipliers; nothing is restated as a literal.

Coefficient-basis toggle. The viewer embeds both coefficient sets per well and switches client-side (D-034). setBasis() sets one BASIS flag read by wB() for the per-well layer and cIB() for the cluster layer, so the two cannot disagree — the mixed state fixed in v2.11.2, where the full-record coefficients never reached the embedded WELLS array. Whole record is the default, matching the report. Button labels and tooltips are built by basis_labels() from fit_start and fit_end in 03_15 rather than typed, so they cannot drift from the fits; an output directory predating those columns still renders, undated, with a warning. Selecting the recent basis raises a note assembled from CLIMATE.sig_counts, because on that basis several clusters — C4 above all — have a drainage term the data cannot distinguish from zero.

Basemap strength. From v2.12.0 the hillshade PNG is emitted opaque and blended at draw time rather than with alpha baked in at encoding, starting at VIEWER_HILLSHADE_ALPHA. The slider (sBg) spans 0 to 2 in steps of 0.05, shown as 0 to 200 per cent. Up to 1 the value is globalAlpha; above 1 the image is drawn again with globalCompositeOperation set to multiply at alpha (value – 1), which darkens the shaded faces without flattening the lit ones. Both globalAlpha and the composite mode are restored afterwards. Quantisation rose from 32 to 64 grey levels at the same time, since banding invisible at low strength shows at full.

Linear features. kml_to_bng already read LineStrings from Features.kml but had nowhere to put them. From v2.13.0 the eighteen linear features are carried as POLYS.tracks and drawn as a separate “Paths and lines” layer, checkbox chkTrk, off by default. They are open polylines and so are drawn directly rather than through dpoly(), which closes its path; the layer is deliberately separate from the KML polygon toggle because the lines are ground reference features, not analysis units.

Part C

Seasonal Extremes Scatter

C1. Overview

The seasonal extremes scatter (14_seasonal_extremes_scatter.html) is generated by Script 14 (14_climate_projections.py, v1.4.1). It reads per-well summary statistics from the well network table (00_well_network_table.csv) and cluster assignments from 02_cluster_stats.csv.

C2. Data Sources

- Mean_Summer_Min_m: mean of annual summer (Apr–Sep) minimum water-table depths, 2005–2026, per well. Values in metres relative to pipe top (negative = below).
- Mean_Winter_Max_m: mean of annual winter (Oct–Mar) maximum depths, computed identically.
- Cluster: from 02_cluster_stats.csv. Unmatched wells labelled “UNKNOWN”.

C3. Threshold Definitions

Ecological thresholds from Curreli et al. (2013), imported from utils/config.py:

Constant	Value (m)	Meaning
SD15b (summer)	0.61	Summer minimum viability limit for wet dune slack.
SD16 (summer)	0.98	Summer minimum viability limit for dry dune slack.
SD15b_WINTER	0.10	Winter maximum flooding threshold for wet slack.
SD16_WINTER	0.25	Winter maximum flooding threshold for dry slack.

C4. Chart Implementation

Uses Chart.js 4.4.1 from cdnjs. Well data serialised as JSON point objects. A custom plugin (thresholdPlugin) draws the four threshold lines after the scatter data. The search mechanism rebuilds datasets with modified colours and radii: matched well in orange (#ff6600) at radius 13, others at 33% opacity.

C5. Cluster Styling

Colours, labels, and markers defined in utils/config.py (CLUSTER_COLOURS, CLUSTER_LABELS, CLUSTER_MARKERS) as the single source of truth. Five clusters under the $k = 5$ partition: C1 Lake Edge, C2 Dune, C3 Western Residual, C4 Main Forest, C5 Coastal Forest.

C6. Build Provenance

Script 14 also produces static PNG figures (summer trajectory, winter flooding, stacked two-panel), a summer trend CSV, annual extremes CSV, and winter exceedance summary. The scatter HTML is an additional interactive output complementing these. tool1